Ticket: CampusCart JPA Product Catalog (DTOs + Postman v1)

Customer: CampusCart catalog screen
Need: Snacks must **list**, **load by id**, and **create** in **MySQL** so a restart does not wipe the menu. JSON is a **DTO** (request / response), not the `Product` table class. POST uses `@Valid`. Invalid body returns a **lite 400** (`error` + `fields`). Ship **Postman collection v1** (this folder). Persistence is still **`Product` + `JpaRepository` + MySQL** — not an `ArrayList`.

Teaching-order: Keep `@Entity` / `JpaRepository` / MySQL list / get / create from the JPA notes. Add `ProductRequest` / `ProductResponse`, **manual** mapping (no MapStruct), `spring-boot-starter-validation`, `@Valid` on POST, `@NotBlank` / `@NotNull` (lite `@Size` / `@Min`), `@RestControllerAdvice` (or `@ControllerAdvice`) for **400** only. Keep **404** on GET by id as already taught. Import `CampusCart_Products_v1.postman_collection.json`. `jakarta.validation` / `jakarta.persistence`. No Security / JWT. No Lombok. CORS for Vite `:5173` is **Stretch** (same notes, React weeks **Later —**).

Each of `starter/`, `broken/`, and `solution/` is a Maven tree. Open the folder that contains **`pom.xml`**.

Starter

Complete TODOs in `Product`, `ProductRepository`, `ProductController`, `application.yml`, then `ProductRequest`, `ProductResponse`, `ProductMapper`, and `ApiExceptionHandler`. Export env from `.env.example` (Boot does **not** load `.env` by itself). `ddl-auto=update` can create `campus_products`; optional SQL is `seed/campus_products.sql`. Import the Postman collection at the kit root.

Broken

`javax.persistence` on the entity (Boot 3 needs `jakarta.persistence`). Password literal in yml. `ddl-auto: create-drop`. Controller still uses an `ArrayList`. Repository id type is `Integer` (must be `Long`). Missing `starter-validation`. JSON is still the entity (no DTO / `@Valid` / advice). Fix until curl **and** the Postman collection match Done when, and a restart still shows POST data.

Seed

`seed/campus_products.sql` — optional INSERTs into `training_db.campus_products`. Does **not** replace MYSQL/ homework tables.

Postman

File: `CampusCart_Products_v1.postman_collection.json` (Collection v2.1). In Postman: **Import** → this file. Variable `baseUrl` = `http://localhost:8080`. Run **Create product** before **Get product by id** if the table is empty.

Run

```
# from kit root
# cp .env.example starter/.env   # optional reminder file; still EXPORT for Boot
export MYSQL_PASSWORD='your_lab_password'
export MYSQL_USER=root
export MYSQL_DATABASE=training_db
```

```
cd starter          # or broken / solution
mvn spring-boot:run
```

Optional seed (after the table exists, or run the SQL which CREATE TABLE IF NOT EXISTS):

```
mysql -u root -p < ../seed/campus_products.sql
```

Second terminal:

```
curl http://localhost:8080/products
curl http://localhost:8080/products/1
curl http://localhost:8080/products/99
curl -X POST http://localhost:8080/products \
  -H "Content-Type: application/json" \
  -d '{"name":"Pani puri","priceInr":30}'
curl -X POST http://localhost:8080/products \
  -H "Content-Type: application/json" \
  -d '{"name":"","priceInr":30}'
# stop the server, mvn spring-boot:run again, then GET /products — Pani puri must remain
```

Invalid POST must be **400** with JSON keys `error` and `fields` — not a new MySQL row.

Done when

- [ ] `mvn spring-boot:run` starts (Tomcat + DataSource; MySQL reachable)
- [ ] `Product` is `@Entity` with `@Id` + `@GeneratedValue` (`jakarta.persistence`)
- [ ] `ProductRepository` extends `JpaRepository<Product, Long>`
- [ ] GET `/products` JSON **array** of **`ProductResponse`** from **`findAll()`** (MySQL, not `ArrayList`)
- [ ] GET `/products/{id}` uses `@PathVariable` + `findById` — **200** `ProductResponse` or **404**
- [ ] POST `/products` uses `@Valid @RequestBody ProductRequest` + map + `save` — **201**; id from MySQL
- [ ] Blank / missing name (or missing price) returns **400** with `error` + `fields` (advice)
- [ ] Restart; POST row is still in GET list
- [ ] `CampusCart_Products_v1.postman_collection.json` imports and the five requests match the statuses above
- [ ] Password is not a literal in yml or source; `.env` not committed
- [ ] No Security, MapStruct, or Lombok required
- [ ] `target/` gitignored

Stretch

- `@CrossOrigin(origins = "http://localhost:5173")` (or a small CORS config) — **not** Done when. Production origins **Later —**.
- Extra field (e.g. `category`) — not Done when.
- HTTP DELETE `/products/{id}` with `deleteById`.
- Lombok getters / Java records — optional, not in Done when.
- **Later —** Spring Security / JWT. **Later —** React `fetch` against this API.